Kernel Methods

Feature maps

- In the last discussion about linear regression for a non-linear function of x We start by considering fitting cubic functions $y = \theta_3 x^3 + \theta_2 x^2 + \theta_1 x + \theta_0$.
- It turns out that we can view the cubic function as a linear function over a different set of feature variables (defined below). Concretely, let the function $\phi: \mathbb{R} \to \mathbb{R}^4$ be defined as

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix} \in \mathbb{R}^4 \qquad (1)$$

- We can rewrite the cubic function in x as:

$$\theta_3 x^3 + \theta_2 x^2 + \theta_1 x + \theta_0 = \theta^T \phi(x)$$

LMS (least mean squares) with features

We will derive the gradient descent algorithm for fitting the model $\theta^T \phi(x)$.

$$\theta := \theta + \alpha \sum_{i=1}^{m} \left(y^{(i)} - h_\theta \left(x^{(i)} \right) \right) x^{(i)}$$

$$:= \theta + \alpha \sum_{i=1}^{m} \left(y^{(i)} - \theta^T x^{(i)} \right) x^{(i)} \qquad (2)$$

LMS (least mean squares) with features

- Let $\phi: \mathbb{R}^d \to \mathbb{R}^p$ be a feature map that maps attribute x (in $\mathbb{R}^d$) to the features $\phi(x)$ in $\mathbb{R}^p$.

- We can replace all the occurrences of $x^{(i)}$ in the algorithm above by $\phi\left(x^{(i)}\right)$ to obtain the new update:

$$\theta := \theta + \alpha \sum_{i=1}^{m} \left(y^{(i)} - \theta^T \phi\left(x^{(i)}\right)\right) \phi\left(x^{(i)}\right) \qquad (3)$$

- Similarly, the corresponding stochastic gradient descent update rule is:

$$\theta := \theta + \alpha \left(y^{(i)} - \theta^T \phi\left(x^{(i)}\right)\right) \phi\left(x^{(i)}\right) \qquad (4)$$

LMS with the kernel trick

- Gradient descent update, or stochastic gradient update becomes computationally expensive when the features $\phi(x)$ is high-dimensional.
- Suppose $x \in \mathbb{R}^d$, and let $\phi(x)$ be the vector that contains all the monomials of x with degree ≤ 3

$$\phi(x) = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_1^2 \\ x_1 x_2 \\ x_1 x_3 \\ \vdots \\ x_2 x_1 \\ \vdots \\ x_1^3 \\ x_1^2 x_2 \\ \vdots \end{bmatrix} \quad (5)$$

$x_{d \times 1}$

$\phi(x)_{d^3 \times 1}$

$\theta_{d^3 \times 1}$

$p = d^3$

$d \rightarrow p$

$x \quad \phi(x)$

LMS with the kernel trick

- The main observation is that at any time, θ can be represented as a linear combination of the vectors $\left(\phi(x^{(1)}), \ldots, \phi\left(x^{(m)}\right)\right)$.
- If we initialize $\theta\, 0 = \sum_{i=1}^{m} 0 \cdot \phi\left(x^{(i)}\right)$. Assume at some point, θ can be represented:

$$\theta = \sum_{i=1}^{m} \beta_i \phi\left(x^{(i)}\right) \qquad (6)$$

$$\underline{\beta} = [\beta_1, \beta_2, \ldots, \beta_m]^T$$

LMS with the kernel trick

for some $\beta_1, \ldots, \beta_m \in \mathbb{R}$. Then we claim that in the next round, θ is still a linear combination of $\phi(x^{(1)}), \ldots, \phi(x^{(m)})$ because:

$$\begin{aligned}
\theta &= \theta + \alpha \sum_{i=1}^{m} \left(y^{(i)} - \theta^T \phi(x^{(i)})\right) \phi(x^{(i)}) \\
&= \sum_{i=1}^{m} \beta_i \phi(x^{(i)}) + \alpha \sum_{i=1}^{n} \left(y^{(i)} - \theta^T \phi(x^{(i)})\right) \phi(x^{(i)}) \\
&= \sum_{i=1}^{m} \underbrace{\left(\beta_i + \alpha \left(y^{(i)} - \theta^T \phi(x^{(i)})\right)\right)}_{\text{new } \beta_i} \phi(x^{(i)}) \qquad (7)
\end{aligned}$$

Using the equation above, we see that the new β_i depends on the old one via

$$\beta_i := \beta_i + \alpha \left(y^{(i)} - \theta^T \phi(x^{(i)})\right) \qquad (8)$$

LMS with the kernel trick

Here we still have the old θ on the RHS of the equation. Replacing θ by $\theta = \sum_{j=1}^{m} \beta_j \phi\left(x^{(j)}\right)$ gives:

$$\forall i \in \{1, \ldots, m\}, \beta_i := \beta_i + \alpha \left(y^{(i)} - \sum_{j=1}^{m} \beta_j \phi\left(x^{(j)}\right)^T \phi\left(x^{(i)}\right) \right)$$

We often rewrite $\phi\left(x^{(j)}\right)^T \phi\left(x^{(i)}\right)$ as $\left\langle \phi\left(x^{(j)}\right), \phi\left(x^{(i)}\right) \right\rangle$ to emphasize that it's the inner product of the two feature vectors.

LMS with the kernel trick

two important properties come to rescue:

1. We can pre-compute the pairwise inner products $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$ for all pairs of i, j before the loop starts.
2. For the feature map ϕ defined in (5) (or many other interesting feature maps), computing $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$ can be efficient and does not necessarily require computing $\phi(x^{(i)})$ explicitly. This is because:

$$\begin{aligned}\langle \phi(x), \phi(z) \rangle &= 1 + \sum_{i=1}^{d} x_i z_i + \sum_{i,j \in \{1,\dots,d\}} x_i x_j z_i z_j + \sum_{i,j,k \in \{1,\dots,d\}} x_i x_j x_k z_i z_j z_k \\ &= 1 + \sum_{i=1}^{d} x_i z_i + \left(\sum_{i=1}^{d} x_i z_i\right)^2 + \left(\sum_{i=1}^{d} x_i z_i\right)^3 \\ &= 1 + \langle x, z \rangle + \langle x, z \rangle^2 + \langle x, z \rangle^3 \end{aligned} \quad (9)$$

LMS with the kernel trick

We define the Kernel corresponding to the feature map ϕ as a function that maps $\mathcal{X} \times \mathcal{X} \to \mathbb{R}$ satisfying:

$$K(x, z) \triangleq \langle \phi(x), \phi(z) \rangle \tag{10}$$

we write down the final algorithm as follows:

1. Compute all the values $K\left(x^{(i)}, x^{(j)}\right) \triangleq \left\langle \phi\left(x^{(i)}\right), \phi\left(x^{(j)}\right) \right\rangle$ for all $i, j \in \{1, \ldots, m\}$.

   Set $\beta := 0$.

2. Loop:

$$\forall i \in \{1, \ldots, m\}, \beta_i := \beta_i + \alpha \left(y^{(i)} - \sum_{j=1}^{m} \beta_j K\left(x^{(i)}, x^{(j)}\right) \right) \tag{11}$$

   Or in matrix-vector notation, letting K be the $m \times m$ matrix with $K_{ij} = K\left(x^{(i)}, x^{(j)}\right)$, we have: $\beta := \beta + \alpha(\vec{y} - K\beta)$

LMS with the kernel trick

With the algorithm above, we can update the representation β of the vector θ efficiently with $O(m)$ time per update. Finally, we need to show that the

- knowledge of the representation β suffices to compute the prediction $\theta^T \phi(x)$.

Indeed, we have:

$$h_\theta(x) = \theta^T \phi(x) = \sum_{i=1}^{m} \beta_i \phi\left(x^{(i)}\right)^T \phi(x) = \sum_{i=1}^{m} \beta_i K\left(x^{(i)}, x\right)$$

Properties of kernels

$$K(x, z) = \langle \phi(x), \phi(z) \rangle$$

$\phi(z)$, $\phi(x)$

کرنل: معیاری برای اندازه‌گیری ... بافت حیان

$k \uparrow \longrightarrow$ ... نزدیکند $\phi(z)$, $\phi(x)$

$$K(x,z) = \exp\left(-\frac{\|x-z\|^2}{2\sigma^2}\right)$$

$x = z \Rightarrow k = 1$

$$K(x, z) = \phi^T(x)\phi(z)$$

کرنل گوسی متناظر با $\phi(x)$ با بعد نامتناهی است.

Necessary conditions for valid kernels

if K is a valid kernel, then

$$K_{ij} = K\left(x^{(i)}, x^{(j)}\right) = \phi\left(x^{(i)}\right)^T \phi\left(x^{(j)}\right) = \phi\left(x^{(j)}\right)^T \phi\left(x^{(i)}\right) = K\left(x^{(j)}, x^{(i)}\right) = K_{ji}$$

and hence K must be symmetric.

Necessary conditions for valid kernels

Moreover, letting $\phi_k(x)$ denote the k-th coordinate of the vector $\phi(x)$, we find that for any vector z, we have

$$
\begin{aligned}
z^T K z &= \sum_i \sum_j z_i K_{ij} z_j \\
&= \sum_i \sum_j z_i \phi(x^{(i)})^T \phi(x^{(j)}) z_j \\
&= \sum_i \sum_j z_i \sum_k \phi_k(x^{(i)}) \phi_k(x^{(j)}) z_j \\
&= \sum_k \sum_i \sum_j z_i \phi_k(x^{(i)}) \phi_k(x^{(j)}) z_j \\
&= \sum_k \left(\sum_i z_i \phi_k(x^{(i)}) \right)^2 \\
&\geq 0
\end{aligned}
$$

باشد PSD باید K

Sufficient conditions for valid kernels

Theorem (Mercer)

$\underline{\underline{K}}_{m \times m}$

Let $K: \mathbb{R}^d \times \mathbb{R}^d \mapsto \mathbb{R}$ be given. Then for K to be a valid (Mercer) kernel, it is necessary and sufficient that for any $\{x^{(1)}, \ldots, x^{(m)}\}, (n < \infty)$, the corresponding kernel matrix is **symmetric positive semi-definite**.

$$k(x,z) = \exp\left(-\frac{(x-z)^T(x-z)}{2\sigma^2}\right)$$

$$k(x,z) = (x^T z)^k$$

چند جمله‌ای

Application of kernel methods

- We've seen the application of kernels to linear regression.
- In the next part, we will introduce the **support vector machines** to which kernels can be directly applied. dwell too much longer on it here
- Specifically, if you have any learning algorithm that you can write in terms of only inner products $\langle x, z \rangle$ between input attribute vectors, then by replacing this with $K(x, z)$ where K is a kernel, you can "magically" allow your algorithm to work efficiently in the high dimensional feature space corresponding to K.

Properties of kernels

- Can we tell if there is some feature mapping ϕ so that $K(x,z) = \phi(x)^T \phi(z)$ for all x, z ?

Suppose $x, z \in \mathbb{R}^d$, and let's first consider the function $K(\cdot,\cdot)$ defined as:

$$K(x,z) = \left(x^T z\right)^2.$$

We can also write this as:

$$\begin{aligned} K(x,z) &= \left(\sum_{i=1}^{d} x_i z_i\right)\left(\sum_{j=1}^{d} x_j z_j\right) \\ &= \sum_{i=1}^{d}\sum_{j=1}^{d} x_i x_j z_i z_j \\ &= \sum_{i,j=1}^{d} (x_i x_j)(z_i z_j) \end{aligned}$$

Properties of kernels

- can we tell if there is some feature mapping ϕ so that $K(x,z) = \phi(x)^T \phi(z)$ for all x, z ?

  Thus, we see that $K(x,z) = \langle \phi(x), \phi(z) \rangle$ is the kernel function that corresponds to the feature mapping ϕ given (shown here for the case of $d = 3$) by

$$\phi(x) = \begin{bmatrix} x_1 x_1 \\ x_1 x_2 \\ x_1 x_3 \\ x_2 x_1 \\ x_2 x_2 \\ x_2 x_3 \\ x_3 x_1 \\ x_3 x_2 \\ x_3 x_3 \end{bmatrix}$$

Properties of kernels

- Revisiting the computational efficiency perspective of kernel, note that whereas calculating the high-dimensional $\phi(x)$ requires $O(d^2)$ time, finding $K(x, z)$ takes only $O(d)$ time-linear in the dimension of the input attributes.
- For another related example, also consider $K(\cdot,\cdot)$ defined by

$$K(x, z) = (x^T z + c)^2$$

$$= \sum_{i,j=1}^{d} (x_i x_j)(z_i z_j) + \sum_{i=1}^{d} \left(\sqrt{2c}x_i\right)\left(\sqrt{2c}z_i\right) + c^2$$

$$\phi(x) = \begin{bmatrix} x_1 x_1 \\ x_1 x_2 \\ x_1 x_3 \\ x_2 x_1 \\ x_2 x_2 \\ x_2 x_3 \\ x_3 x_1 \\ x_3 x_2 \\ x_3 x_3 \\ \sqrt{2c}x_1 \\ \sqrt{2c}x_2 \\ \sqrt{2c}x_3 \\ c \end{bmatrix}$$

Kernels as similarity metrics

$$f_1 = \text{similarity}(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right) = \exp\left(-\frac{\sum_{j=1}^{n}(x_j - zj)^2}{2\sigma^2}\right)$$

If $x \approx z$

$$\exp\left(-\frac{0}{2\sigma^2}\right) \approx 1$$

$a_0 + a_1(x - z) + a_2(x - z)^2 + \cdots$

If x is far from z

$$\exp\left(-\frac{(large\ number)}{2\sigma^2}\right), \approx 0$$

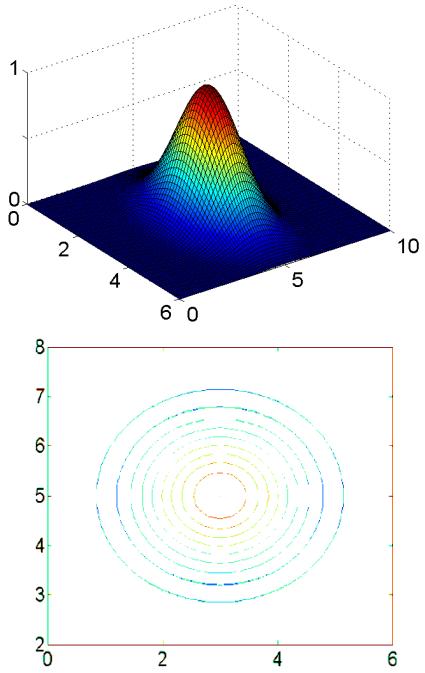
Gaussian Kernel corresponding to an infinite dimensional feature mapping Φ

Kernels as similarity metrics

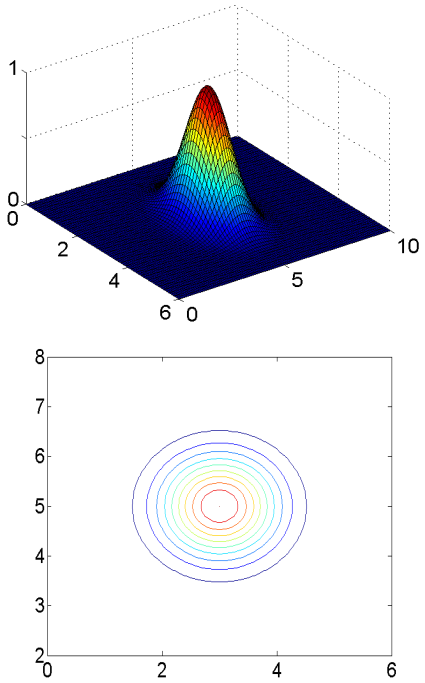
Example:

$$l^{(1)} = \begin{bmatrix} 3 \\ 5 \end{bmatrix}, \quad f_1 = \exp\left(-\frac{\|x - l^{(1)}\|^2}{2\sigma^2}\right)$$

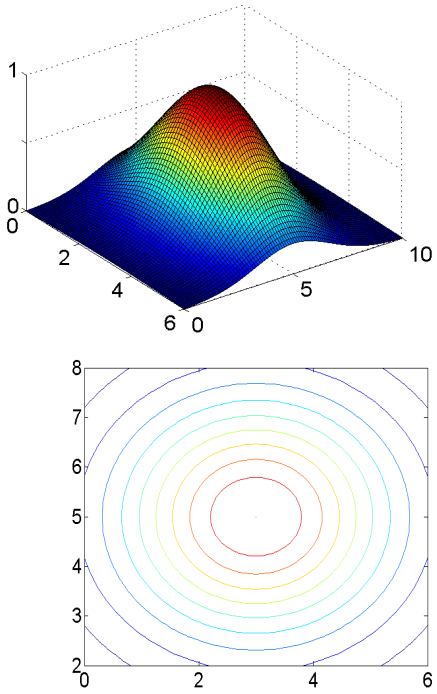
$\sigma^2 = 1$



$\sigma^2 = 0.5$



$\sigma^2 = 3$



Support Vector Machines

Margins:

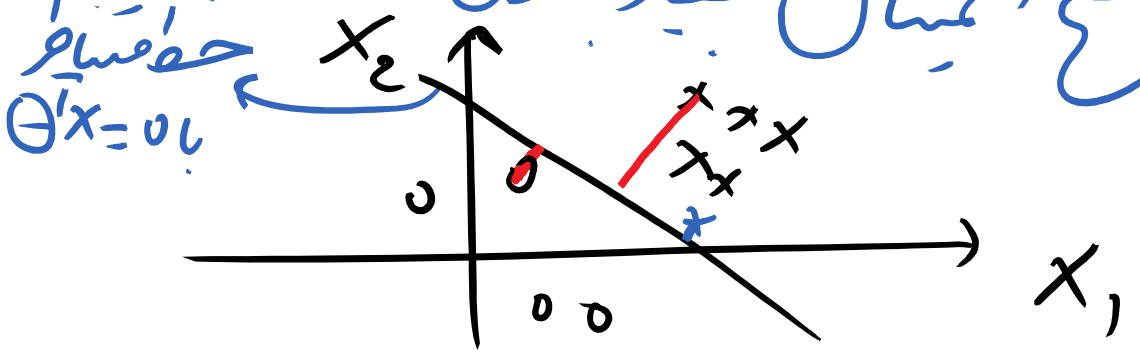
logReg $h_\theta(x) \geq 0.5 \equiv \theta^T x \geq 0 \Rightarrow y = 1$

$\theta^T x < 0 \Rightarrow y = 0$

اگر مدل به درستی کار کند f_{i+} برای $y^{(i)} = 1$ $\Leftarrow$ $\theta^T x^{(i)} \gg 0$

$(x^{(i)}, y^{(i)})$

$\Leftarrow$ سطح اطمینان بالایی دارد $\theta^T x^{(i)} \ll 0$ ، $y^{(i)} = 0$



Hyperplane : $w_0 + w_1 X_1 + w_2 X_2 + \dots w_p X_p = 0$

(فضای p بعدی)

$w_0 + w_1 X_1 = 0$ خط

$w_0 + w_1 X_1 + w_2 X_2 = 0$ صفحه

$w_0 + w_1 X_1 + \dots + w_p X_p > 0 \Rightarrow y = 1$

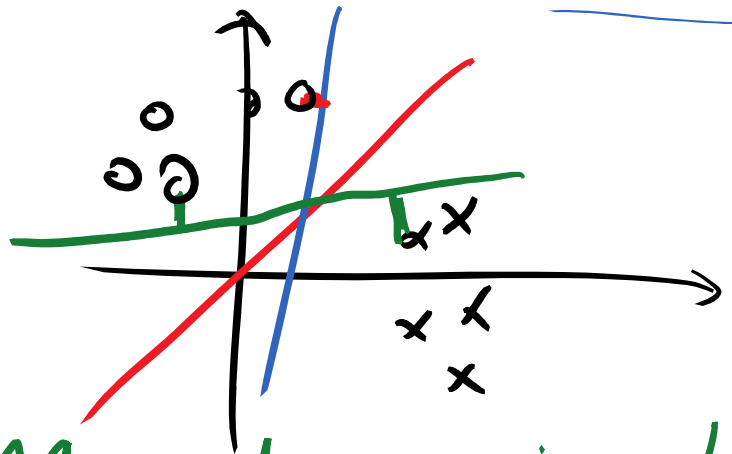
$< 0 \Rightarrow y = 0$

فضای p بعدی را به دو قسمت تقسیم می‌کند.

$y_i \in \{-1, 1\}$ Binary classification

$$y^{(i)}\left(w_1 x_1^{(i)} + w_2 x_2^{(i)} + \cdots w_n x_n^{(i)}\right) > 0$$

$i = 1 \ldots m$



$\{x^{(i)}, y^{(i)}\}$

$i = 1 \ldots m$

Maximal margin classifier:

- $X^{(i)}$ از هایپرپلین Margin کوتاه‌ترین فاصله عمودی مشاهده‌های

حاشیه:

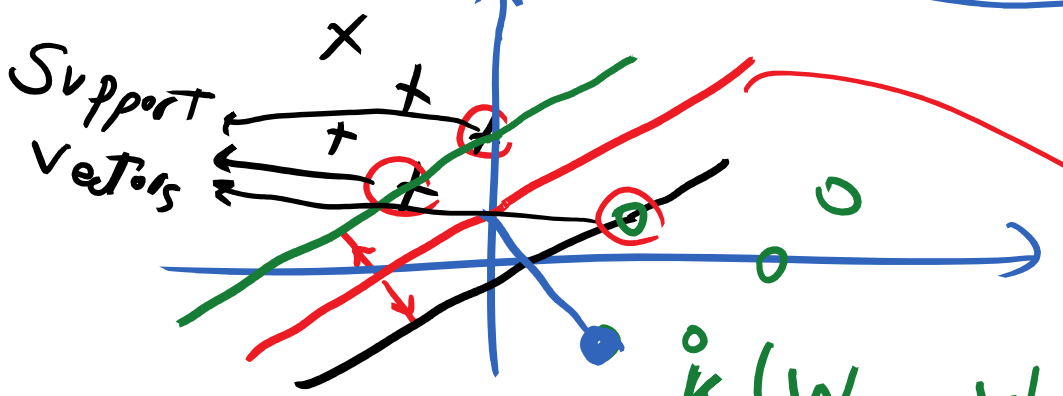
L M M classifier

$$\max_{W_0, W_1, \dots, W_n, M} M$$

subject to $\sum_{j=1}^{n} W_j^2 = 1$

$$y^{(i)}\left(W_0 + W_1 x_1^{(i)} + \dots + W_n x_n^{(i)}\right) \geq M$$

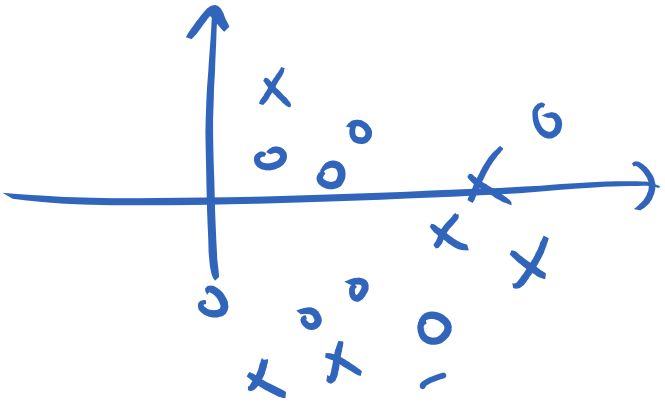
$\forall i = 1 \dots m$



MMH

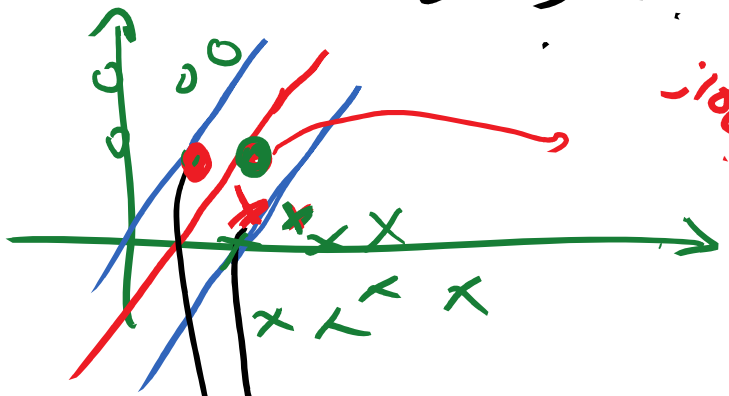
$$k(W_0 + W_1 X_1 + \dots + W_n X_n) = 0$$

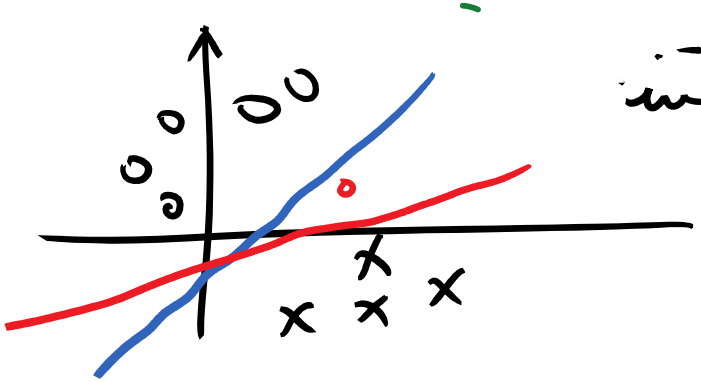
2_ Support vector classifier:



۱_ برای داده‌های جدید مقاوم است

۲_ جداسازی بهتری برای اکثر مشاهدات صورت می‌دهد.





margin حاشیه

بردار پشتیبان

hyperplane

Support vector classifier:

$$\max_{w_0, w_1, \dots, w_n, \epsilon_1, \epsilon_2, \dots, \epsilon_m, M} M$$

Subject to $\sum_{j=1}^{n} w_j^2 = 1$

$$y^{(i)}(w_0 + w_1 x_1^{(i)} + \dots + w_n x_n^{(i)}) \geq M(1 - \epsilon_i)$$

$$\epsilon_i \geq 0 \qquad \sum_{i=1}^{m} \epsilon_i \leq C \qquad C > 0$$

C: عددی دلخواه و مثبت

hyperplane, margin

MMC ← C = 0

- $x^{(i)}$ روی مرز margin است ← $\epsilon_i = 0$
- $x^{(i)}$ ... ← $\epsilon_i > 0$
- $x^{(i)}$ ... hyperplane ← $\epsilon_i > 1$

$C \uparrow \rightarrow$ Margin $\uparrow$

$C \uparrow \rightarrow$ bias $\uparrow$, var $\downarrow$

$C \downarrow \rightarrow$ bias $\downarrow$, var $\uparrow$

فقط نقاطی که روی حاشیه (مرز) قرار می‌گیرند یا از آن تجاوز می‌کنند تعیین کننده hyperplane هستند ← بردارهای پشتیبان (SV)

C ↑ ⇒ تعداد SV ↑ ، واریانس ↓

C ↓ ⇒ ↓ ، ، ، ، bias ↓

3- Support vector Machine

SV classifier + Kernel

$x \longrightarrow \phi(x)$

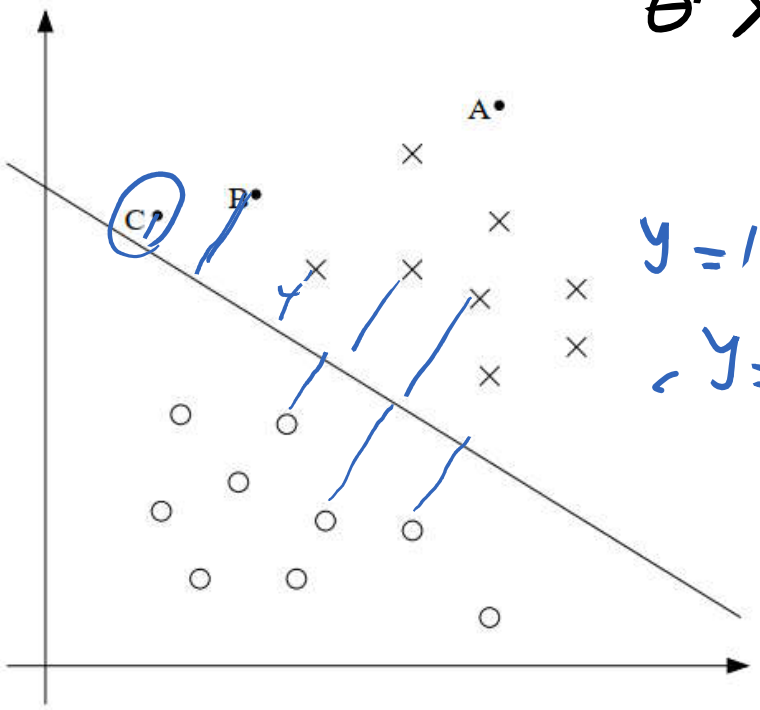
Margins

Logistic Regression $h_\theta = g(\theta^T x) \geq .5 \rightarrow y = 1$

$< .5 \rightarrow y = 0$

$\theta^T x \geq 0 \rightarrow y = 1$

$< 0 \rightarrow y = 0$



$y = 1 \quad \theta^T x \gg 0$

$y = 0 \quad \theta^T x \ll 0$

Hyperplane : $w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$

$\theta^T X = \theta_n x_n + \theta_{n-1} x_{n-1} + \dots + \theta_1 x_1 + \theta_0 x_0 = 0$

$\begin{bmatrix} 1 \\ x_1 \\ x_n \end{bmatrix}$

$\theta^T X = w^T X + b$

$\begin{bmatrix} x_1 \\ x_n \end{bmatrix}$

$w^T = [\theta_1 \dots \theta_n]$ | $b = \theta_0$

$x_2 + 2 x_1 = 0$

